Laura Hietala

Oppimistehtävä 2

Tietojenkäsittelyn koulutus
TKMI23KV

14.11.2023

Sovellusohjelmointi 1
Janne Turunen

Tehtäväraportti

Tehtävänä on toteuttaa React-tekniikoin sovellus, jolla voi pitää kirjaa henkilökohtaisista veloista ja saatavista. Sovellusta käyttämällä käyttäjä tietää/muistaa keneltä on lainannut rahaa ja vastaavasti kenelle on itse lainannut rahaa. Vippikirja saa "tyhjentyä" uudelleenkäynnistyksen yhteydessä. Tietoja ei siis tarvitse tallentaa palvelimelle, localStorageen tms.

GitHub*:* *https://github.com/xamk-so1/oppimisteht-v-2-elahi001xamk*

Käytän esimerkkinä pääasiassa Velat-taulukkoa, sillä toiminnallisuudet molemmissa taulukoissa ovat samoja.

Perusvaatimukset sovellukselle

Vaatimus: Aloitusnäkymässä on kaksi taulukkoa (table) "Velat" tai "Saatavat". Molemmissa taulukoissa on kaksi saraketta: Henkilön nimi (kenelle on lainattu tai keneltä laina on) ja lainasumma. Jokainen rivi vastaa yhtä velkaa tai saatavaa.

Ratkaisu: Riviltä 74 alkaa taulukko ”Velat”. Se jatkuu riville 119 saakka ja se sisältää kaksi sellaista saraketta, joille tulee jotain dataa (rivit 78-79 aloittavat nämä sarakkeet otsikoilla ”Kenelle” ja ”Summa”). Rivit 86-93 ja 94-102 määrittelevät input-sarakkeet, joilla on mahdollista syöttää tietoa taulukkoon. Tiedot tulevat sarakkeisiin input-kenttien alle sitä mukaa, kun tietoa syötetään. Sama rakenne on toteutettu myös "Saatavat" taulukolle.

```
74    <table>
75      <thead>
76        <h2>Velat</h2>
77          <tr>
78            <th>Kenelle</th>
79            <th>Summa</th>
80            <th>Toiminto</th>
81          </tr>
82      </thead>
83
84      <tbody>
85      <tr>
86            <td>
87              <input
88                name="nimi"
89                placeholder="Nimi"
90                value={velkaNimi}
91                onChange={(e) => setVelkaNimi(e.target.value)}
92              />
93            </td>
94            <td>
95              <input
96                name="summa"
97                placeholder="Summa"
98                value={velkaSumma}
99                onChange={(e) => setVelkaSumma(Number(e.target.value))}
100               onKeyDown={handleKeyDownVelka}
101             />
102           </td>
103           <td>
104             <button onClick={lisääVelka}>Lisää velka</button>
105           </td>
106         </tr>
107
108       {velat.map((velka, index) => (
109       <tr key={index}>
110
111         <td>{velka.nimi}</td>
112         <td>{velka.summa}</td>
113         <td>
114           <button onClick={() => poistaVelka(index)}>Poista</button>
115         </td>
116     </tr>
117     ))}
118     </tbody>
119   </table>
```

Vippikirjanpito

Velat

Kenelle	Summa	Toiminto
Nimi	0	Lisää velka
Aku Ankka	313	Poista
Iines Ankka	131	Poista

Velkasi on yhteensä 444 euroa.

Saatavat

Keneltä	Summa	Toiminto
Nimi	0	Lisää saatava
Touho Ankka	120	Poista
Tupu Ankka	21	Poista

Saatavasi on yhteensä 141 euroa.

Vaatimus: Käyttöliittymässä on mahdollisuus uuden velan/saatavan lisäämiseen. Tarvittavat tiedot (nimi ja summa) kysytään lomakkeella ja viedään ao. taulukkoon uudeksi riviksi nappia painamalla. Lomake voi sijaita osana käyttöliittymää esim. taulukoiden ala- tai yläpuolella. Erillistä dialogi-ikkunaa tai vastaavaa ei tarvitse toteuttaa.

Ratkaisu: Uuden velan tai saatavan lisääminen on toteutettu lomakkeella, joka sisältää kaksi input-kenttää "Nimi" ja "Summa" (käytän nyt esimerkkinä Velat-taulukkoa, jossa nämä ovat riveillä 87-92 ja 95-101, kuva ylempänä). Kun käyttäjä syöttää tiedot ja painaa "Lisää velka" tai "Lisää saatava" -nappia, uusi rivi lisätään vastaavaan taulukkoon. Rivin 80 value={velkaNimi} asettaa kentän arvoksi React-tilamuuttujan nimiarvon. Rivin 98 value={velkaSumma} asettaa kentän arvoksi React-tilamuuttujan summa-arvon.

Rivillä 91 onChange={(e) => setVelkaNimi(e.target.value)} määrittelee tapahtumankäsittelijän, joka päivittää React-tilamuuttujan *nimi* uudella arvolla aina, kun käyttäjä muuttaa kentän sisältöä. Rivillä 99 onChange={(e) => setVelkaSumma(Number(e.target.value))} päivittää React-tilamuuttujan *summa* aina, kun käyttäjä muuttaa kentän sisältöä. Number(e.target.value) varmistaa, että syöte muutetaan numeroksi. Rivillä 100 onKeyDown={handleKeyDownVelka} (joka on määritelty riveillä 24-29) kutsuu määriteltyä tapahtumankäsittelijää, kun käyttäjä painaa nappia, joka määritellään rivillä 104. Tällä rivillä onClick={lisääVelka} kutsuu lisääVelka-funktiota, kun käyttäjä painaa nappia. Kun käyttäjä täyttää Nimi- ja Summa-kentät ja painaa "Lisää velka" -nappia, uusi velka lisätään taulukkoon. Tämä tapahtuu lisääVelka-funktion (rivit 17-22) kautta, joka päivittää tilamuuttujat ja päivittää näkymän.

```
16
17  const lisääVelka = () => {
18    const uusiVelka = { nimi: velkaNimi, summa: velkaSumma };
19    setVelat([...velat, uusiVelka]);
20    setVelkaNimi('');
21    setVelkaSumma(0);
22  };
23
24  const handleKeyDownVelka = (event: React.KeyboardEvent<HTMLInputElement>) => {
25    if (event.key === "Enter") {
26      event.preventDefault();
27      lisääVelka();
28    }
29  };
30
```

Vaatimus: Käyttöliittymässä tulee näkyä selvästi velkojen ja saatavien kokonaissummat taulukoiden ala- tai yläpuolella. Esim: "Velkasi on yhteensä xx euroa" ja "Saatavasi on yhteensä xx euroa". Luonnollisesti kokonaissummat päivittyvät, kun uusi tieto lisätään.

Ratkaisu: Kokonaissummat näkyvät käyttöliittymässä taulukoiden alapuolella ja päivittyvät automaattisesti, kun uusi velka tai saatava lisätään. Nämä näytetään riveillä 121 (velat) ja 172 (saatavat) olevissa TSX-elementeissä:

```
120
121     <h5>Velkasi on yhteensä {velat.reduce((total, velka) => total + velka.summa, 0)} euroa.</h5>
122
```

```
171
172       <h5>Saatavasi on yhteensä {saatavat.reduce((total, saatava) => total + saatava.summa, 0)} euroa.</h5>
173
```

Rivien 57-67 useEffect-hookit seuraavat tilamuuttujien muutoksia ja suorittavat niiden vaikutuksia. Ne laskevat velkojen ja saatavien kokonaissummia ja tulostavat ne konsoliin.

```
57  useEffect(() => {
58  const velkaTotal = velat.reduce((total : any, velka : any) => total + velka.summa, 0);
59
60  console.log ('Velkasi on yhteensä', velkaTotal);
61  }, [velat]);
62
63  useEffect(() => {
64  const saatavaTotal = saatavat.reduce((total : any, saatava : any) => total + saatava.summa, 0);
65
66  console.log ('Saatavasi on yhteensä', saatavaTotal);
67  }, [saatavat]);
```

Toisaalta tajusin tätä kirjoittaessa, että en ehkä olisi tarvinnut useEffectiä lainkaan, sillä laskentaa ja päivittämistä suoritetaan myös TSX-elementeissä eikä tarkoitus ole esim. tallentaa nyt mitään.

Lisävaatimukset sovellukselle

Vaatimus: Taulukoista voidaan poistaa mikä tahansa rivi (velka tai saatava). Jokaisella rivillä on painonappi, jota klikkaamalla rivi poistetaan. Rivin poisto voi tapahtua välittömästi, eli poiston varmistusta ei tarvitse toteuttaa. Yhteissummat päivittyvät poistaessa.

Ratkaisu: Jokaisella rivillä taulukoissa on "Poista" -nappi, jonka avulla käyttäjä voi poistaa kyseisen velan tai saatavan. Yhteissummat päivittyvät automaattisesti poiston jälkeen. Nappi velkojen poistamiseen määritellään rivin 114 TSX-

elementissä. Saatavien poistamiseen on luonnollisesti myös oma nappinsa.

```
113                 <td>
114                   <button onClick={() => poistaVelka(index)}>Poista</button>
115                 </td>
116               </tr>
```

Riveillä 45-49 ja 51-55 määritellään nappien painamiseen sidotut muuttujat (poistaVelka ja poistaSaatava). Ne huolehtivat velkojen ja saatavien poistamisesta. Käytän taas Velka-osiota esimerkkinä (Saatavissa tapahtuu sama asia): Ensin luodaan veloista kopio uusiVelka spredaamalla (rivi 46). Rivin 47 splice-metodi poistaa taulukosta yhden alkion indeksinumeron avulla. Rivillä 48 päivitetään tila uudella taulukolla, josta yksi alkio on poistettu.

```
45  const poistaVelka = (index : number) => {
46    const uusiVelka = [...velat];
47    uusiVelka.splice(index, 1);
48    setVelat(uusiVelka);
49  };
50
51  const poistaSaatava = (index : number) => {
52    const uusiSaatava = [...saatavat];
53    uusiSaatava.splice(index, 1);
54    setSaatavat(uusiSaatava);
55  };
```